实验报告

一、链表类模板源码以及注解

1、Node.h

```
#ifndef NODE_H_INCLUDED
#define NODE_H_INCLUDED

template <class T>
class Node{
private:
    Node<T> *next;//指向后继结点的指针
public:
    T data;//数据域

    Node(const T & data,Node<T> *next=0);//构造函数
    void insertAfter(Node<T> *p);//在本结点之后插入一个同类结点 p
    Node<T> *deleteAfter();//删除本结点的后继结点，并返回其地址
    Node<T> *nextNode();//获取后继结点的地址
    const Node<T> *nextNode() const;//获取后继结点的地址
};


//构造函数，初始化数据和指针成员
template <class T>
Node<T>::Node(const T & data,Node<T> *next/*=0*/):data(data),next(next){}//对于默认值的声明，只要在定义的时候声明就行  连续声明两次会报错 redeclaration

//获取后继结点的地址
template <class T>
Node<T> * Node<T>::nextNode()
{
    return next;
}

//获取后继结点的地址
template <class T>
const Node<T> * Node<T>::nextNode() const
{
    return next;
}

//在本结点之后插入一个同类结点 p
```

```
template <class T>
void Node<T>::insertAfter(Node<T> *p)
{
	p -> next=this -> next;//p 指针指向当前结点后一个结点
	this -> next=p;//当前结点指向 p 结点
}

//删除本结点的后继结点，并返回其地址
template <class T>
Node<T> * Node<T>::deleteAfter()
{
	Node<T> * tempPtr = this -> next;//临时指针指向想要删除的结点
	if (this -> next=0)//如果当前结点没有后继结点，返回空指针
		return 0;
	this -> next=tempPtr -> next;//使当前结点指针指向 Temp 的后继结点，从而达到删除一个结点的操作
	return tempPtr;//返回删除结点的地址
}
#endif // NODE_H_INCLUDED
```

2、LinkedList.h

```
#ifndef LINKEDLIST_H_INCLUDED
#define LINKEDLIST_H_INCLUDED

#include "Node.h"

//front->a1->a2->...->rear

template <class T>
class LinkedList {
private:
	//数据成员
	Node<T> *front, *rear;//表头和表尾指针
	Node<T> *prePtr,*currPtr;//记录表当前遍历位置的指针，由插入和删除操作更新
	int size;//表中的元素个数
	int position;//当前元素在表中的位置序号(游标位置 currPtr)
	//函数成员
	//生成新结点，数据域为 item，指针域为 ptrNext
	Node<T> *newNode(const T & item,Node<T> *ptrNext=nullptr);

	//释放结点
```

```
        void freeNode(Node <T> *p);

        //将链表 L 复制到当前表（假设当前表为空）
        //被复制构造函数、operator=调用
        void copy(const LinkedList<T> & L);

public:
        LinkedList();//构造函数
        LinkedList(const LinkedList<T> & L);//复制构造函数
        ~LinkedList();//析构函数
        LinkedList<T> & operator=(const LinkedList<T> & L);//重载赋值运算符

        int getSize() const;//返回链表中元素个数
        bool isEmpty() const;//链表是否为空

        void reset(int pos=0);//初始化游标位置
        void next();//使游标移动到下一个结点
        bool endOfList() const;//游标是否到了链表尾
        int currentPosition() const;//返回游标当前位置

        void insertFront(const T & item);//在表头插入结点
        void insertRear(const T & item);//在表尾添加结点
        void insertAt(const T & item);//在当前结点之前插入结点
        void insertAfter(const T & item);//在当前结点之后插入结点

        T deleteFront();//删除头结点
        void deleteCurrent();//删除当前结点

        T & data();//返回对当前结点成员数据的引用
        const T & data() const;//返回对当前结点成员数据的常引用

        void clear();//清空链表：释放所有结点的内存空间，被析构函数、operator=
调用
};


template <class T>
//生成新结点，数据域为 item，指针域为 ptrNext
Node<T> * LinkedList<T>::newNode(const T & item,Node<T> *ptrNext)
{
        Node<T> *tempPtr=new Node<T>(item,ptrNext);//为临时指针申请一个空间，
其 data 为 item，next 为 ptrNext
        return tempPtr;
}
```

```
template <class T>
//释放结点
void LinkedList<T>::freeNode(Node<T> *p)
{
	delete p;
}

template <class T>
//将链表 L 复制到当前表（假设当前表为空）
void LinkedList<T>::copy(const LinkedList<T> & L)
{
	Node<T> *temp=L.front,*ptr=front;//声明并定义赋值为头指针
	temp=temp -> nextNode();//将 temp 指向第一个结点
	while (temp!=L.rear)//遍历，除尾结点
	{
		ptr -> next=new Node<T>(temp -> data,nullptr);//为下一个结点申请空间
		ptr=ptr ->next;//当前结点指向下一个结点
		temp=temp -> next;
	}
	ptr -> next= rear;//指向尾结点
	size=L.getSize();
	position=L.currentPosition();
}

template <class T>
//构造函数
LinkedList<T>::LinkedList()
{
	rear=new Node<T>(0,nullptr);
	front=new Node<T>(0,rear);//=0，front->rear
	currPtr=rear;
	prePtr=front;
	size=0;//不计算 front 和 rear
	position=0;//在 front 下一个元素视为 0
	std::cout<<" Call Constructor"<<std::endl;
}

template <class T>
//复制构造函数
LinkedList<T>::LinkedList(const LinkedList<T> & L)
{
	rear=new Node<T>(0,nullptr);
	front=new Node<T>(0,rear);
```

```
        currPtr=rear;
        prePtr=front;
        size=0;
        position=0;
        copy(L);//复制链表
        std::cout<<"Call Copy constructor"<<std::endl;
}

template <class T>
//析构函数
LinkedList<T>::~LinkedList()
{
        clear();//清除链表
        delete front;//释放所申请的内存
        delete rear;
}

template <class T>
//重载复制运算符
LinkedList<T> & LinkedList<T>::operator=(const LinkedList<T> & L)
{
        clear();
        copy(L);
        std::cout<<"Call Operator Overloading"<<std::endl;
}

template <class T>
//返回元素个数
int LinkedList<T>::getSize() const
{
        return size;
}

template <class T>
//链表是否为空
bool LinkedList<T>::isEmpty() const
{
        return (size == 0);//当链表为空时返回 1，不为空返回 0
}

template <class T>
//初始化游标位置
void LinkedList<T>::reset(int pos)
{
```

```
	if (pos>size)//判断游标是否大于总长度
	{
		std::cout<<"越界，无法访问"<<std::endl;
		return;
	}
	prePtr=front;
	currPtr=front -> nextNode();
	for (int i=0;i<pos;i++)//遍历，将 currPtr 指向游标位置
	{
		prePtr=currPtr;
		currPtr=currPtr -> nextNode();
	}
	position=pos;
}

template <class T>
//使游标移动到下一个结点
void LinkedList<T>::next()
{
	if	(currPtr == rear)
	{
		std::cout<<"已经到达链表尾，无法移动"<<std::endl;
		return ;
	}
	prePtr=currPtr;
	currPtr=currPtr -> nextNode();
	position++;
}

template <class T>
//游标是否到了链表尾
bool LinkedList<T>::endOfList() const
{
	return (currPtr == rear);//如果已经到达链表尾返回 1，没有返回 0
}

template <class T>
//返回游标当前位置
int LinkedList<T>::currentPosition() const
{
	return position;
}

template <class T>
```

```
//在表头插入结点
void LinkedList<T>::insertFront(const T & item)
{
	Node<T>*temp=newNode<T>(item,front-> nextNode());
	//temp->data=item,temp->next=front->next
	front=new Node<T>(0,temp);//为头指针申请新的内存空间，并指向当前插入的点
	size++;
	position++;
}

template <class T>
//在表尾添加结点
void LinkedList<T>::insertRear(const T & item)
{
	Node<T> *p1=front,*p2=front ->nextNode();
	while (p2!=rear)//遍历，最终 p1 指向最后一个结点，p2 指向 rear
	{
		p1=p2;
		p2=p2 -> nextNode();
	}
	Node<T> *temp= new Node<T>(item,rear);
	p1 -> insertAfter(temp);//在 p1 之后插入一个同类结点 temp，指向 rear
	size++;
}

template <class T>
//在当前结点之前插入结点
void LinkedList<T>::insertAt(const T & item)
{
	Node<T>  *temp=  new  Node<T>(item,currPtr);//声明一个临时指针指向当前结点
	prePtr -> insertAfter(temp);//在前一个结点之后插入该新结点
	size++;
}

template <class T>
//在当前结点之后插入结点
void LinkedList<T>::insertAfter(const T & item)
{
	Node<T>  *temp=  new  Node<T>(item,currPtr  ->  nextNode());//声明一个临时指针指向当前结点后一个结点
	currPtr -> insertAfter(temp);//在当前结点后插入该新结点
	size++;
```

```
}

template <class T>
//删除头结点
T LinkedList<T>::deleteFront()
{
	Node<T> *temp=front -> nextNode();//声明一个临时指针指向头结点
	temp=temp -> nextNode();//将临时指针指向第二个结点
	delete front;//释放原本头结点内存
	front= new Node<T>(0,temp);//为头指针申请一个新的内存，并指向第二个
结点
	size--;
	return temp->data;//返回当前头指针的数据域
}

template <class T>
//删除当前结点
void LinkedList<T>::deleteCurrent()
{
	Node<T> *temp=prePtr -> deleteAfter();//声明一个临时指针指向需要删除的
节点
	currPtr=temp -> nextNode();
	delete temp;
	size--;
}

template <class T>
//返回对当前结点成员数据的引用
T & LinkedList<T>::data()
{
	return currPtr -> data;
}

template <class T>
////返回对当前结点成员数据的引用
const T & LinkedList<T>::data()const
{
	return currPtr;
}

template <class T>
//清空链表：释放所有结点的内存空间
void LinkedList<T>::clear()
{
```

```
    Node<T> *temp=front -> nextNode();
    while (temp!=rear)//从头结点开始遍历，
    {
        front=temp;
        delete temp;
        temp=front -> nextNode();
    }
    size=0;
    position=0;
}

#endif // LINKEDLIST_H_INCLUDED
```

3、Main.cpp

```
#include <iostream>
#include "LinkedList.h"

using namespace std;

int main()
{
    LinkedList<int> list;

    //输入 10 个整数依次向表头插入
    for (int i=0;i<10;i++)
    {
        int item;
        cin>>item;
        list.insertFront(item);
    }

    //输出链表
    cout<<"List:";
    list.reset();//将游标位置挪到头结点

    //输出各结点数据，直到链表尾
    while (!list.endOfList())
    {
        cout<<list.data()<<" ";
        list.next();
    }
    cout<<endl;
```

```
//输出链表元素个数
cout<<"Size:"<<list.getSize()<<endl<<endl;

//输入需要删除的结点
int key;
cout<<"Please enter some integer needed to be deleted:";
cin>>key;

//查找并删除结点
list.reset();
while (!list.endOfList())
{
	if (list.data()==key)
		list.deleteCurrent();
	list.next();
}

//输出当前链表
cout<<"List:";
list.reset();

//输出各结点数据，直到链表尾
while (!list.endOfList())
{
	cout<<list.data()<<" ";
	list.next();
}
cout<<endl;

//输出当前链表元素个数
cout<<"Size:"<<list.getSize()<<endl<<endl;

//删除头结点
cout<<"Delete the first integer:"<<list.deleteFront()<<endl;

 //输出当前链表
cout<<"List:";
list.reset();

//输出各结点数据，直到链表尾
while (!list.endOfList())
{
	cout<<list.data()<<" ";
```

```
            list.next();
        }
        cout<<endl;

        //输出当前链表元素个数
        cout<<"Size:"<<list.getSize()<<endl<<endl;

        //在表头添加一个新结点
        cout<<"Please enter the integer wanted to be added to the front:";
        cin>>key;
        list.insertFront(key);

        //输出当前链表
        cout<<"List:";
        list.reset();

        //输出各结点数据，直到链表尾
        while (!list.endOfList())
        {
            cout<<list.data()<<" ";
            list.next();
        }
        cout<<endl;

        //输出当前链表元素个数
        cout<<"Size:"<<list.getSize()<<endl<<endl;

        //输出当前结点
        cout<<"Current:"<<list.data()<<endl;

        //在当前结点之后插入新结点
        cout<<"Please enter the integer wanted to be added to the front of the current
one:";
        cin>>key;
        list.insertAt(key);

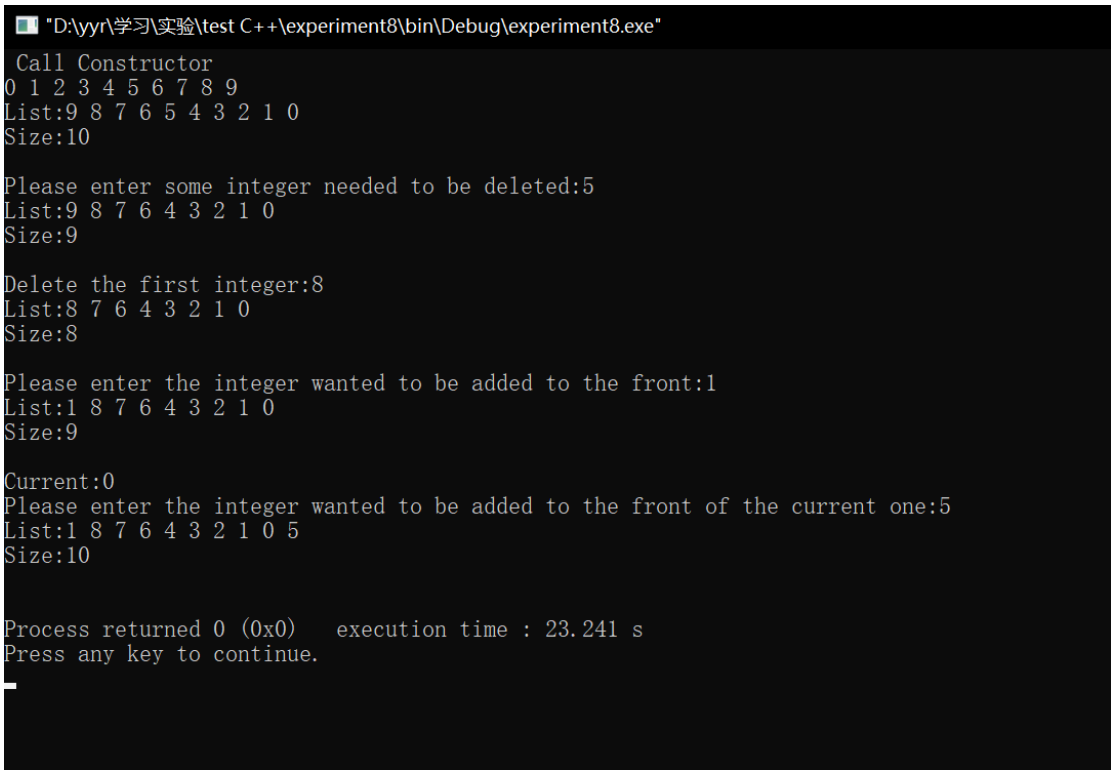
        //输出当前链表
        cout<<"List:";
        list.reset();

        //输出各结点数据，直到链表尾
        while (!list.endOfList())
        {
            cout<<list.data()<<" ";
```

```
            list.next();
        }
        cout<<endl;

        //输出当前链表元素个数
        cout<<"Size:"<<list.getSize()<<endl<<endl;

        return 0;
}
```



二、Debug

1、类模板中的成员函数的默认参数，必须在第一次声明时给出，若在函数实现时给出，或在声明时以及实现时都给出，编译器会报错。错误如下：

error: redeclaration of ‘XXX’ may not have default arguments [-fpermissive]|

2、next 在 Node.h 头文件中定义为 private 类型的数据成员，因此属于被保护起来的对象，只能在类中的成员函数中使用，不可在类

外使用，因此在实现 LinkedList 中的函数时，不可通过实例化的对象直接调用 next，需要通过调用 Node 中 public 类型的成员函数获取 next 的内容，即需使用 ptr -> nextNode()，而不是 ptr -> next。错误如下：

error: 'Node<int>* Node<int>::next' is private within this context|

3、ptr -> nextNode() 只能作为右值，不可作为左值。因此若需要在等号左侧使用 next，只能通过成员函数实现（如：通过 Node 中定义的 insertAfter（）直接在当前结点之后插入结点，无需再 prePtr -> next = temp)。错误如下：

error：lvalue required as left operand of assignment